COURSE NAME:
Object Oriented Programming Lab (OOPs Lab)

PROJECT TITLE:
Library Management System

STUDENT NAME:
Diyali Das

ROLL NUMBER:
62

SECTION:
A

DEPARTMENT:
Information Technology (IT)

INSTRUCTOR NAME:
Dr. Soumadip Biswas
Dr. Susovan Jana

ACADEMIC SESSION:
Spring 2026

1. Problem Statement

In many educational institutions and public libraries, the management of books and user transactions is still carried out using manual or semi-digital methods. These traditional approaches involve maintaining physical registers or basic spreadsheet records to track information such as book availability, user details, issue dates, return dates, and fines. While these methods may be sufficient for small-scale operations, they become increasingly inefficient, error-prone, and difficult to manage as the size of the library and the number of users grow.

One of the major challenges associated with manual library systems is the lack of an efficient mechanism to track the availability of books in real time. Librarians often need to manually search through records to determine whether a particular book is available or currently issued. This process is time-consuming and may lead to incorrect information being provided to users. As a result, users may face unnecessary delays and inconvenience when trying to access required resources.

Another critical issue is the improper handling of borrowing and return policies. In manual systems, due dates are often recorded without any automated reminders or validation, increasing the likelihood of missed deadlines. Additionally, the calculation of fines for overdue books is typically performed manually, which can result in inconsistencies and errors. This not only

affects the accountability of users but also leads to potential financial discrepancies for the library.

Furthermore, maintaining accurate and organized records of multiple users and their borrowing histories becomes increasingly complex over time. The absence of a centralized and structured system makes it difficult to retrieve past records, generate reports, or analyze usage patterns. This limits the ability of the library administration to make informed decisions regarding resource allocation and policy improvements.

Searching for books based on specific criteria such as title, author, or category is another area where manual systems fall short. The process is often slow and dependent on the librarian's knowledge, which can lead to inefficiencies and reduced user satisfaction. In addition, there is a higher risk of data loss or damage in physical record-keeping systems.

To overcome these challenges, there is a clear need for an automated and efficient solution in the form of a Library Management System (LMS). The proposed system is designed to streamline the core operations of a library by leveraging the capabilities of Java programming and object-oriented principles. It focuses on implementing a structured and reliable mechanism for managing book records, user information, and transaction details.

A key aspect of this system is the implementation of a well-defined borrow and return policy. The system ensures that each issued book is associated with an issue date and a predefined due date. It also automates the process of tracking overdue books and calculating fines based on the number of delayed days. This not only improves accuracy but also enforces discipline and accountability among users.

The system aims to provide the following functionalities:

- Real-time tracking of book availability
- Efficient management of user records
- Automated handling of borrowing and returning processes
- Accurate calculation of fines for late returns
- Easy retrieval and maintenance of transaction records

By addressing the limitations of manual systems, the proposed Library Management System enhances operational efficiency, reduces human errors, and improves the overall user experience. It also serves as a practical application of object-oriented programming concepts in solving real-world problems, making it both technically significant and functionally effective.

2. System Design

The Library Management System is designed using a modular and structured approach to ensure scalability, maintainability, and ease of implementation. The system follows a layered architecture consisting of presentation, business logic, and data handling components.

The presentation layer is implemented as a console-based interface that allows users to interact with the system through a menu-driven approach. It provides options for adding books, viewing records, borrowing books, and returning books. This layer ensures simplicity and ease of use for the end user.

The business logic layer forms the core of the system and is responsible for implementing the rules and policies of the library. It includes functionalities such as checking book availability, assigning due dates, managing borrowing limits, and calculating fines for late returns. This layer ensures that all operations follow predefined rules and constraints.

The data handling layer is responsible for storing and managing data related to books, users, and transactions. In this implementation, data is stored using dynamic data structures such as ArrayLists, which allow efficient storage and retrieval of information during program execution.

The system is divided into several modules, including Book Management, User Management, Transaction Management, and Fine Calculation. Each module is designed to handle specific tasks independently while interacting with other modules when required. This modular design improves code organization and makes the system easier to extend in the future.

3. OOP Concepts Demonstrated

The project effectively demonstrates the use of Object-Oriented Programming (OOP) concepts, which are essential for building scalable and maintainable software systems.

Encapsulation is achieved by defining classes such as Book, User, and Transaction, where data members are kept private and accessed through public methods. This ensures data security and controlled access.

Abstraction is implemented by hiding complex operations behind simple method interfaces. For example, the process of borrowing a book involves multiple steps such as checking availability and updating records, but it is presented as a single method to the user.

Inheritance can be incorporated to extend the functionality of the system. For instance, different types of users such as Admin and Student can inherit from a common User class, allowing code reuse and better organization.

Polymorphism is demonstrated through method overloading or overriding, enabling flexibility in how methods are implemented and used. This allows the system to handle different scenarios efficiently.

Overall, the use of OOP principles ensures that the system is modular, reusable, and easy to maintain.

4. Implementation Highlights

The system is implemented using Java, which provides strong support for object-oriented programming and robust application development. The development process involves creating multiple classes to represent real-world entities such as books, users, and transactions.

The Book class stores details such as book ID, title, author, and availability status. The User class maintains user-related information, while the Transaction class records details of borrowed books, including issue dates and due dates.

The borrow functionality checks whether a book is available before issuing it to a user. If the book is available, it updates the status and records the transaction along with the issue date and due date. The return functionality updates the return date and calculates any applicable fine based on the delay.

Fine calculation is implemented using a simple formula based on the number of days a book is returned late. This ensures fairness and consistency in enforcing library policies.

The system uses ArrayLists to store and manage collections of books, users, and transactions dynamically. This allows efficient handling of data without the need for complex database integration at the initial stage.

5. Testing

Testing is an essential phase in the development process to ensure that the system functions correctly and meets the specified requirements. Various testing techniques are applied to validate different components of the system.

Unit testing is performed on individual methods such as borrowing, returning, and fine calculation to ensure that each function works as expected. Integration testing is conducted to verify that different modules interact correctly with each other.

System testing is carried out to evaluate the overall performance of the application. This includes testing the complete workflow from adding books to borrowing and returning them.

Several test cases are considered, such as borrowing an available book, attempting to borrow an unavailable book, returning a book on time, and returning a book after the due date. These test cases help identify and resolve potential issues in the system.

6. Error Handling

The system incorporates proper error handling mechanisms to ensure smooth execution and prevent unexpected failures. Input validation is performed to ensure that users provide valid data, such as correct book IDs and user IDs.

Exception handling is implemented using try-catch blocks to handle runtime errors gracefully. This prevents the program from crashing and provides meaningful error messages to the user.

Logical error handling is also included to prevent invalid operations, such as borrowing a book that is already issued or returning a book that was not borrowed. Clear and informative messages are displayed to guide the user in such cases.

These measures enhance the reliability and robustness of the system.

7. Git Workflow

Version control is maintained using Git, which helps in tracking changes and managing the development process efficiently. The project repository is initialized using Git, and all files are added and committed with appropriate messages.

Branches are created for implementing new features or making changes without affecting the main codebase. Once the changes are tested, they are merged into the main branch.

Common Git commands used in the project include initializing the repository, adding files, committing changes, creating branches, and merging them. This workflow ensures better organization and collaboration during development.

8. Conclusion

The Library Management System developed in this project successfully automates the core operations of a library, including book management, user management, and transaction handling. The implementation of a borrow and return policy with fine calculation ensures proper regulation and accountability.

The system reduces manual effort, minimizes errors, and improves efficiency. It also demonstrates the practical application of object-oriented programming concepts in solving real-world problems. Overall, the project provides a reliable and effective solution for modern library management.

9. Future Scope

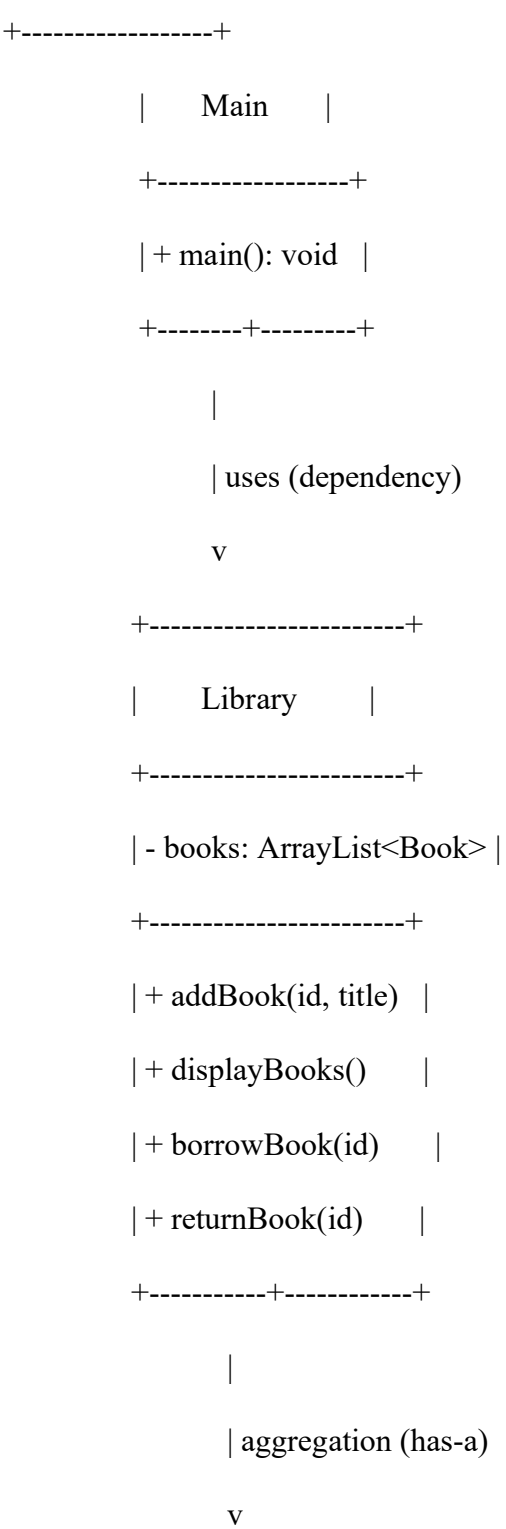
The system can be further enhanced by incorporating advanced features and technologies. A graphical user interface can be developed using Java Swing or JavaFX to improve user interaction.

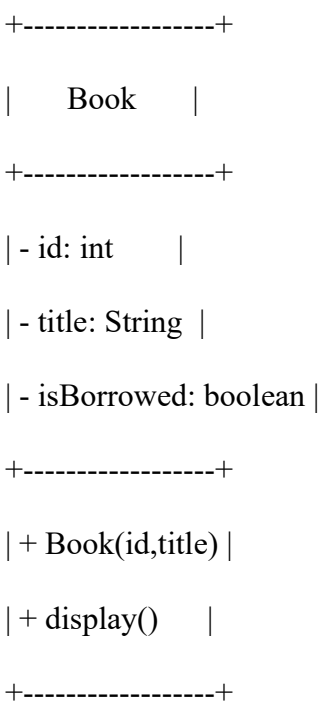
Database integration using MySQL or SQLite can be implemented to store data permanently and handle larger datasets. Additional features such as user authentication, online book reservation, and automated notifications for due dates can also be added.

The system can be extended into a web-based application using frameworks such as Spring Boot, making it accessible from multiple devices. Integration with barcode or QR code systems can further improve efficiency in managing books.

These enhancements will make the system more powerful, scalable, and suitable for real-world deployment.

UML DESIGN:[REPRESENTATION]




Book
- id: int
- title: String
- isBorrowed: boolean
+ Book(id,title)
+ display()